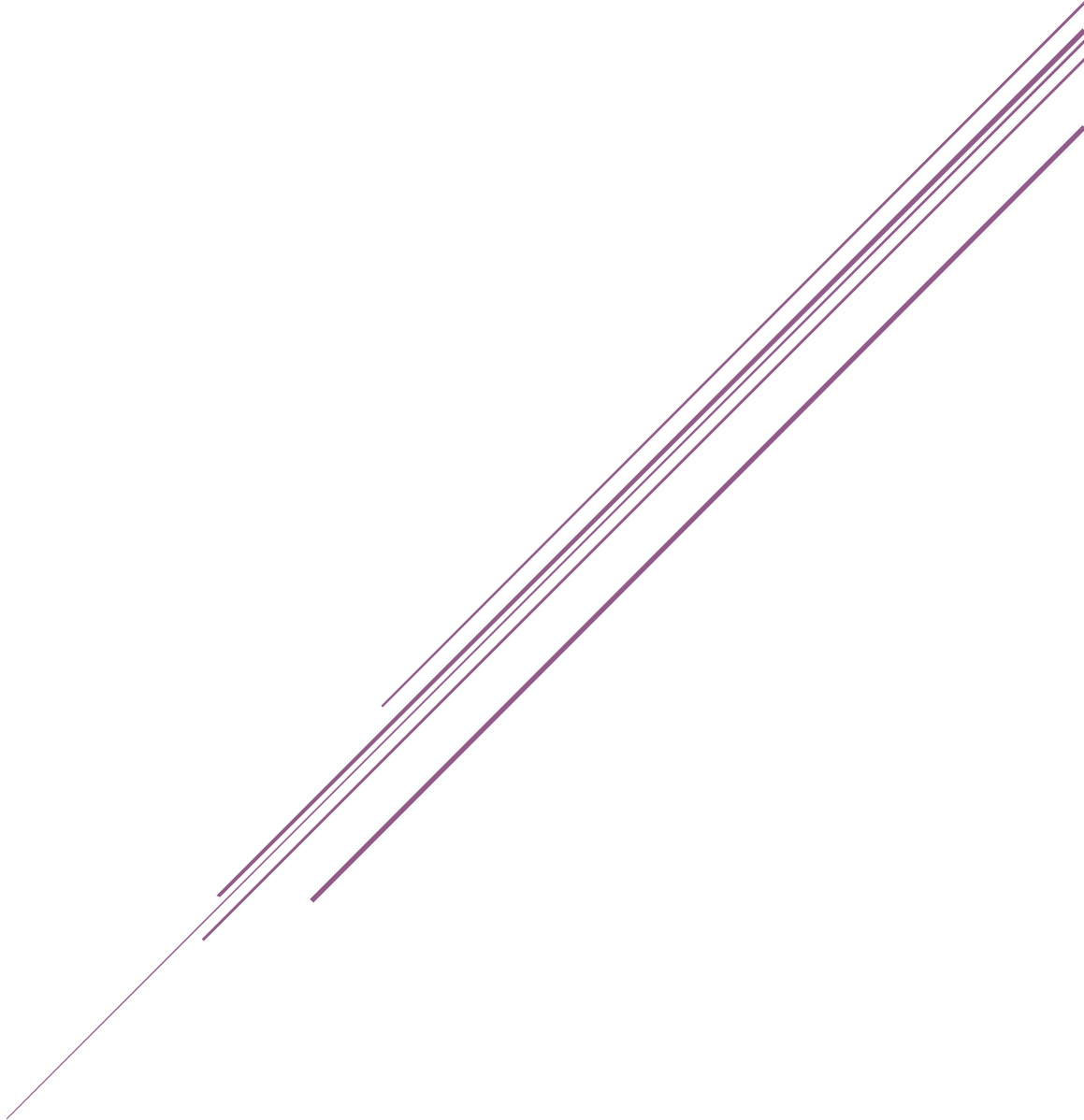


CHES TEACHING TOOL

A LEVEL COMPUTER SCIENCE NEA



Oliver Abbott 3827
Harington School 25345

CONTENTS

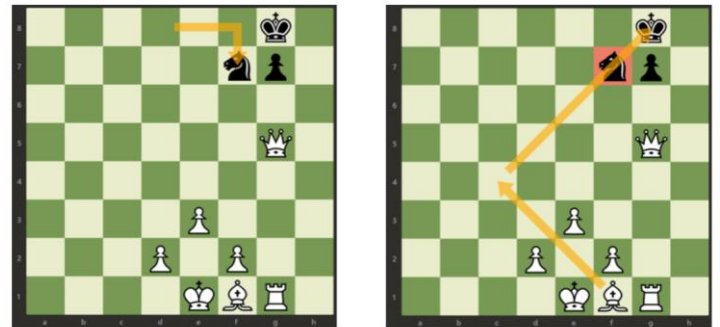
Analysis.....	2
Context of Problem	2
Interview.....	2
Existing System.....	2
Requirements.....	2
Acceptable Limitations	2
Success Criteria	2
Qualitative Criteria.....	2
Quantitative Criteria	2
Documented Design	3
Description.....	3
Interface Design	3
Algorithms.....	3
Technical Solution	4
Testing	5
Test Table.....	5
Evidence.....	5
Evaluation	6
Analysis of Criteria.....	6
End User Feedback	6
Final Thoughts.....	6

ANALYSIS

CONTEXT OF PROBLEM

My secondary school from year 7 to 11 with around 1100 students. They have an elective on Wednesday afternoons after lunch they go to a lesson they pick from art, sports or cultural one of these electives is a chess elective ran by Mr Russel. Currently he uses a set of power points to show the current moves and positions. Some problems with that include not being able to see letters and number position on the board as the screen shots use chess.com with grey on black that are hard to see at the end of a class room and the reason he does not use chess.com's website teaching tool is that the school has a very strong firewall that means chess.com cannot be logged into while on the schools network. This means Mr Russel requires an offline tool that can import and export files from the interface and while the files are in use and displayed they need to be able to be interacted with in clear visible text and be able to move pieces, flip the board, show valid moves and draw an arrow to show where the pieces can move. As show in the screen shot of one of the PowerPoint slides on the right. It is hard to demonstrate all the moves you can do and you can't easily see the checkmate white can make with the queen with this move or the place you should move the queen on the grid as the grid values are small and hard to see.

The PIN



ANALYSIS OF PROBLEM

The problem that currently exists is that chess.com can't be opened or used on the school's network due to this we cannot use the teaching tools on chess.com and thus not be able to move the pieces, draw arrows and set up the boards and explore moves the student wants to try. The PowerPoints are too inconvenient and have far too small text to read easily for the board position when you are not right next to the screen and you can only look at preset moves already on the power point. This is a problem like with the chess.com you cannot explore students moves which is not good for teaching and it is harder for people with people not sitting at the front of the class when they want to participate as they cannot easily read the positions for where they want the piece to go to try a move.

INTERVIEW

My end user will be Mr Russel a computer science teacher for GCSE and A-levels who runs and teaches the chess elective.

Q: What's the problems you are facing currently?

A: "Chess.com current cannot be use on are schools network as it is blocked from inside the school and PowerPoints currently don't work that well as we cannot explore moves that student suggest and the power points are hard to see when you are not sitting at the front of the classroom so students at the back cannot easily tell me what moves they are thinking as they cannot see the grid positions as chess.com is designed for online learning even if we could use it in school is not easy for students in the classroom to learn through it."

Q: What do you want to do with this tool?

A: "I would like the tool to be able to work fully offline that I can load and save boards in a friendly way that is easy to work with. I also would like it to be able to draw arrows to show where the pieces can go and when you move a piece, I would like it to show if that move is valid or not with something visible to teach how pieces can move. I also need to easily be able to see both sides of the boards from a button."

Q: What are the requirements to teach with the tool?

A: "The tool needs to be easy to see from the back of the classroom so the students can see the board and the placement at the back of the class and I need to easily load and select files as well as create them from the software and I need to have most of the functionality of sites like chess.com but offline so I can use them while on the school network and easily demonstrate the moves that can be played."

Q: What are other tools that you have tried and why have they failed for you?

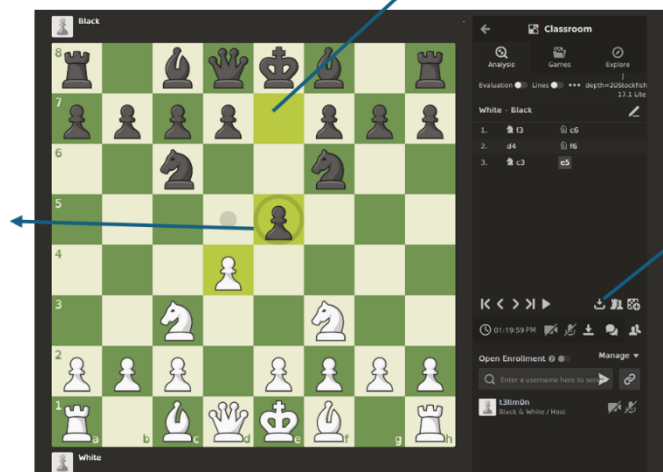
A: "I tried to use chess.com but as discussed earlier it was not a good fit for me due to the school's computer network blocking it. Then I moved onto trying and what I current use is power points but that is not favourable as the text is very small and hard to read from the back and we cannot play moves the students have selected but are not already done."

EXISTING SYSTEM

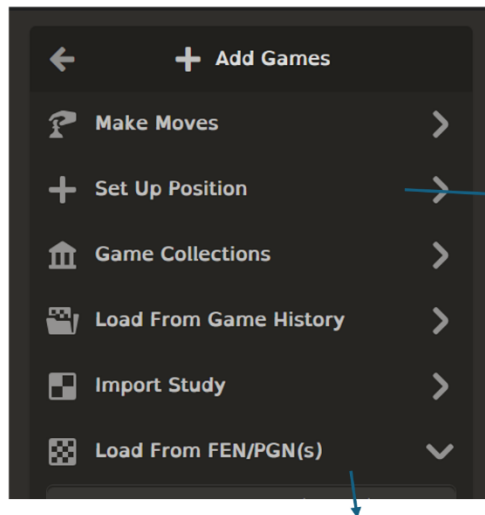
Chess.com

The board is quite easy to see. With pieces visible. From the back and the board is the main part of the program which is easy to see. Which would be good for the back of the board.

You can quite easily see where the white pawn can move in this position. This is useful to the student as you can see where the pawn can move if you are knew to the game.

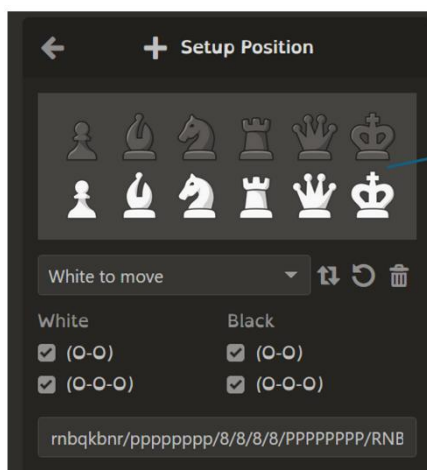


This download button makes it easy to export the board for later and easily import it again for ease of use and make life easier for the teacher so they don't have to re make the board each time.



The button on the side makes it easy for the user to easily access each side and for this to set up the position in this board while on the application. Pressing this button spawns a windows panel that can be used to set up and put pieces on the board. This is convenient so the teacher I does not need to leave the app in order to set up board positions.

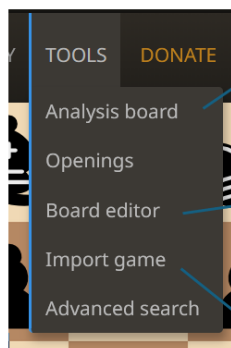
This button brings up a menu to be able to load a board for FEN or PGN notation which is useful to store board states in and easy to edit. FEN show the board it is in now while PEN shows the way it took to get there. So all the moves. FEN is probably easier to implement. Both standards are popular so in this app its easy to import board from other examples using this format.



This is the menu that pops up when you hit Setup Position. This is a little button menu for you to easily set up a board which is quite convenient for the user as to make a board to learn from all they need is to press this button and put the pieces in their places on the board.

Lichess

Analysing board is quite useful as you want to know why that move is good or bad and what it can lead to while you are being taught. Which is good for teaching as when the teacher needs to explain it better or don't quite understand it themselves, they can get here and get more info on the board and that move.



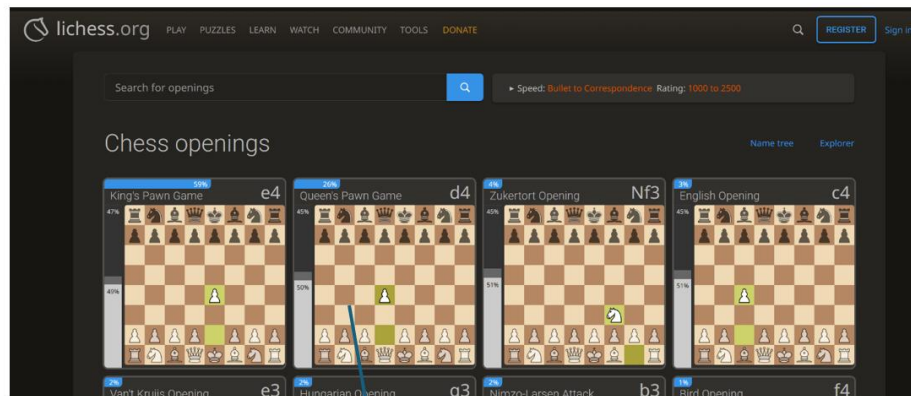
This is used to edit the board position easily which is quite good to make new boards to learn from and export boards to save them for later to use

This is very useful for importing game states to the board to easily teach other concepts in chess like pinning by simply importing a board from a file on your computer.



The board is quite easy to see and is at the centre of the board and is very visible to people in the classroom. Which is quite easy for students to see from the back of the class but there are no grid references on the board which is the only downside I see of it.

This gives a very detailed look of the move you have made and makes life easier for the teacher as they can rely on this to help explain why this move is good and give some backstory to the move and what it can lead to and get down to the grip of what this move archives. This also helps the students who can use this to help understand what that moves are and why it is a good move.



This board makes life quite easy for the teacher easily making them pick openings that are well known to help teach the students and Lichess also has this for mid games and end games which makes it amazing to pick already made moves to help teach and show them off. This is used quite good when you need something quick to teach and its all already organised and students can also go on here as well to learn as well on there own and carry on where the teacher left off.

Real Chess Board

A real chess board is great as you can clearly see the pieces up close and its 3D so you can see it from lots of angles and make it easily see moves with physical pieces. The main problem with a physical board though is that it is quite small so can only really be used for a small class else most people will not be able to see the small board and you would need to remember all the pieces for the board states.



You can quite easily see the pieces of the board and what they look like and can more easily interact with them making it easier to learn and understand how each piece moves.

Although the pieces look nice and are easy to work with if you lose some pieces, it will be a pain as you can no longer use the board to its full effectiveness.

IDENTIFICATION OF USERS

Teachers will be using my system the most. Their technology level would be quite good so there is no need to baby proof the application files systems creation. They will be using the tool to teach the students in the elective chess. This will impact the size and structure of my graphical user interface which means it must be big enough so see far away and have easily accessible buttons to show what the teacher is doing and the program will be used on multiple device with different screen sizes for creation of boards and to be put on the big screen the windows must be scaled with multipliers and not used numbers for a one case screen.

REQUIREMENTS

From the interview. The program needs an easy interface to interact with so teachers can use it quite easily to teach and it is easy for the students to understand when looking at it. The program also would need a way to import and export board files to use in the program so the teacher can easily load new board states to teach from easily and for that we also need a way to create a board inside the program so you can export that board to the computer for it to be imported later. Based on the interview we need to make it easier to see what moves need to be made we also need a way to draw arrows onto the board to easily demonstrate what moves can be made from the position. As this program is going to be used to teach, based on the existing system analysis and interview process, we need a clearly visible and easy to read for the grid references for the students to read at the back of the classroom. As well as this is a teaching tool we need to easily be able to see the opponent's side of the board easily so we would need a switch button so the program can easily switch to the other persons perspective of the board so students can easily see what the other person is looking at and moves they can make for this we also need a way to see if a move is valid. So, for the program there needs to be a signifier to show if a move is valid at that position so the new students to chess can easily see if that move is valid.

ACCEPTABLE LIMITATIONS

My main limitations will be with the checking the validity of the pieces movements due to there being some special case moves like castling and en passant which need special use cases to check their validity including a variable for the queen and rook as well as the positions of the pawn which I don't think is reasonable for this level of project as you would be needing some fixes that are unreasonable for a A-Level project as you would need a way to store if the queen has moved and if the rook has moved and move two pieces at once while checking if both the spaces next to it are free and the en passant will also be a limitation I am willing to put in place as you need to check the position of the pawn that about to be taken and if it moved 2 squares as its last move and the position of the pawn which would take a lot of work for a small feature so I will not be adding it and will make that moves also not in my version and due to the pawn having so many conditions for if any move is valid like if the not moved they can move two squares forward and only being able to take sideways I will not check if any of thoughts are valid and just let it say yes due to the complication of checking the moves.

I also think it is unrealistic to be importing using a ready used format like FEN or PGN as for PGN for example you will need to play through the whole game to get the current board state which is a whole project on its own and would be unrealistic and for FEN notation it has a lot of unneeded information at the end as we are not going to implement the castling feature or any of the other rules that are quite advance for beginners like the half move clock but I do not think it is unrealistic to implement the way it stores the pieces so I will be using the FEN notation but without the extra bits at the end as I think that much is unreasonable for me to implement and is an acceptable limitation for the project.

My last acceptable limitation will be drawing more than one arrow at a time as this requires me to store all the positions of the arrow in a list of data which is hard to keep track of as well as drawing all of them onto the board. I think this would get quite messy quickly and be very complicated to implement so I will be only adding

a way to draw one arrow at a time and making it so that the until the arrow is reset it will use the same start position to make the arrow code more simple and easier to implement for the project and for the same reason when I flip the board I will not be flipping the arrow as well due to the complexity it will take.

SUCCESS CRITERIA

1. Show a chess board
2. loaded images as the pieces with the colour of the pieces visible in the images loaded
3. The pieces must be able to move with user input
4. Moves must be show as valid or invalid using the acceptable limitations as a guide
5. A clear and easy to read area for the grid references to be read from on the side of the board
6. line up to each square of the board to the grid position
7. The top part of the program has buttons to import and export a board
8. The export and input must be files stored in the user's computer.
9. Flip the board with a button
10. Change the sides of the board with the references to flip is to from A-H to H-A
11. Change the sides of the board with the references to flip is to from 1-8 to 8-1
12. Board must be flipped in where the board is reflected in the line $y = x$ and then $y = -x + 8$ so the board is fully reflected. $y = 0$ and $x = 0$ at the first square
13. When draw arrow is pressed change its buttons colour
14. Draw an arrow on the board when draw arrow is pressed.
15. The arrow must be clearly visible above the board without the board disappearing completely
16. The arrow must be draw on top of the board itself.
17. Use the first button as the start of the arrow
18. Use the second button when the end of the arrow is
19. When the user clicks a new button while draw arrow is active change the new end to the new location
20. Remove the arrow and forget it position when draw arrow is re pressed
21. The import button must pop open a window to access the user's files graphicly
22. The user can select the file they want to import
23. Error handling for anything unexpected with reading or writing a file.
24. Read the file
25. Parse the file in to get the board state

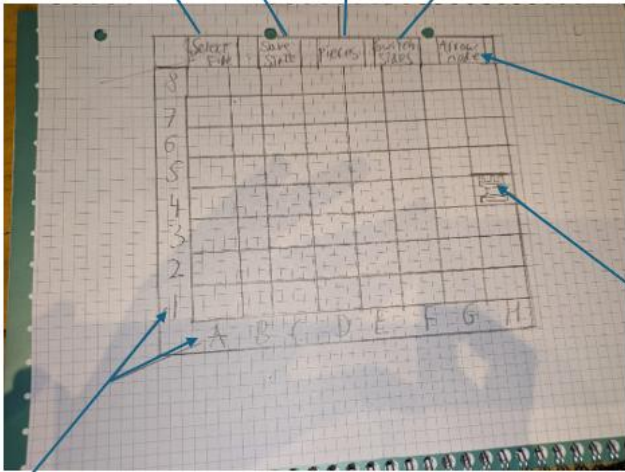
26. Use the imported board state to put it onto the board using text labels text labels.
27. Load images for each for the pieces
28. Load the parsed information onto board
29. The export button must be able to read the board
30. The board must be read into an array to save into the file.
31. The user must be propped with a graphicly way to save the file
32. The file must be saved in the directory selected by the user
33. The file also must be saved with the user selected name
34. The file must then be opened and written into with the data gotten from the array for the board
35. Any errors must be handled and displayed to the user.

DOCUMENTED DESIGN

DESCRIPTION

To solve the problem, I will make an offline chess teaching tool that can import and export chess boards from the program and make it so you can make a chess board inside the board state and store it in a human readable file. I will also make it so the grid references can be read easily from afar and you can easily flip the board with one click of a button and another button to draw an arrow onto the board easily to easily demonstrate the moves. There will also be a way to see if the last move is valid or not for the piece that moved.

INTERFACE DESIGN



This will bring up the menu show in bellow with the grids with the pieces in that when you click the button it will make a pop-up window show with each piece shown that the user can click and then click a piece on the board put a piece onto the board.

Save and select file launch a window show in the sketch below where you can select the file and enter a file name as well.

When you click this button, the board will flip sides so it will look like what the opposite will see. The grid references on the side of the board.

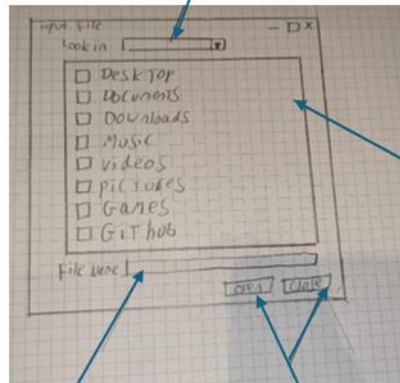
When you click the button, the arrow will turn a different colour. Then the user will click a button on the board then a second button. An arrow will be drawn between the two squares with the first button being the start. If the user presses another square the end of the arrow will change to that square. The arrow will disappear and not come back when arrow mode has been re pressed and the colours changed back

This is the board with a piece on it. The board will be a 2D array of JButtons with a length of 8 and 8 to store the whole grid. The buttons will store a background colour and load images from a file to draw the pieces using icons onto the board. To move a piece simply select the square it is on then select the square you want to move it too. When the piece is moved you need to check if that move is valid and show the square it moved to green or red depending on if the move is valid or not. You can store the pieces onto the JButtons by storing them onto text and then setting the foreground to nothing and the text can not be seen.

The letters and numbers will be stored on in two different JPanel with each letter and number being its own JLabel. Using a grid layout so the board can easily be resized while the letters and numbers stay in alignment to the board. There also will be a way to flip the board simply by changing the text on the JLabel to be reverse so when switched sides is pressed the Side of the board can also switch easily.

File Selector

This is a directory selector which when you go into you can select the directory you want to go into it.



This is a graphic way to select the file you want to use. Select the file or the folder by double clicking to go into it

Type the file you want to save the board as or to the file you want to select to open.

When you click open for the save file you will get the text from this menu and create a new file called the name inputted and save the board to it.

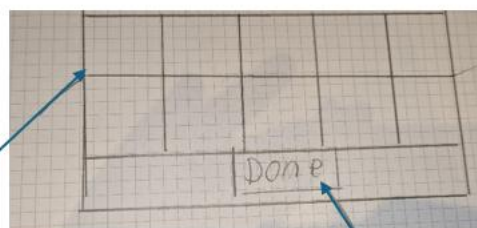
For select file after you press open it will get the name selected and open it and then read the file selected and parse the info into an array to put into the board.

The close button will close the window safely and will skip over reading and writing into a file

Menu for Pieces

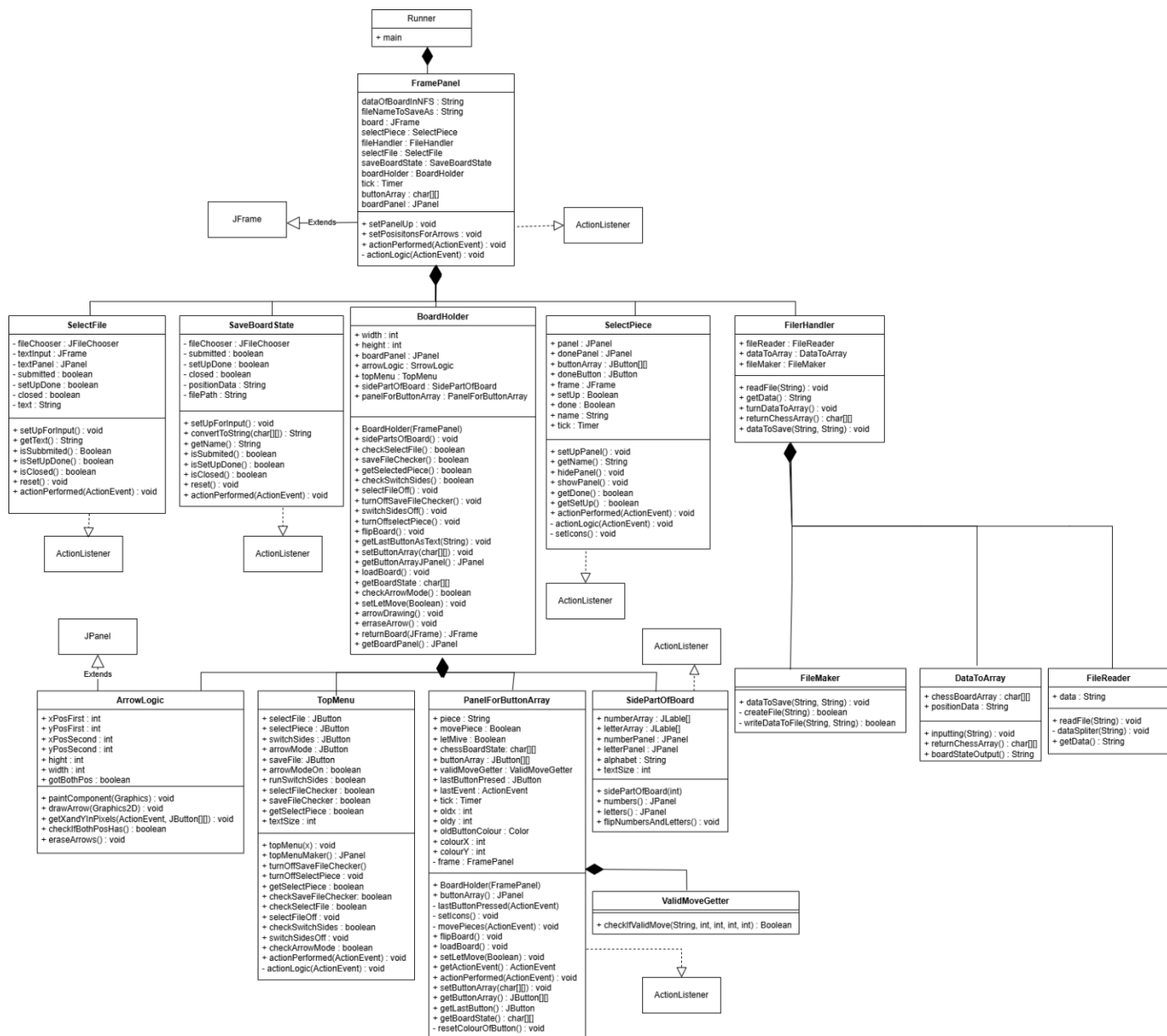
Here will be each piece show for each colour so when the user clicks a piece from here it will then go onto the board where that user next click is.

If the user clicks a new piece on the menu the other think that changes is the users selected piece.



The done button will close the window properly so the board can no longer be added to until the Pieces button is clicked again.

CLASS STRUCTURE



ALGORITHMS

CLASS: RUNNER

This class runs the program

METHOD: main

Creates a frame Panel

CLASS: FramePanel

This class runs holds the JFrame used to display everything and is used to move data to each class needed.

METHOD: FramePanel

This method sets up framePanel loading a map and creating the board using the subclasses functions.

METHOD: setUpPanel

This method sets adds the button array of the map and uses subclasses to create the board and the other buttons in the panel which displays everything the user sees

METHOD: setPosisitonsForArrows

Checks if the arrow button has been pressed if so, it makes it so the pieces can't move and lets the arrow drawing if the arrow button is false then it erased the arrow and sets let move as true.

Check whether arrow mode is enabled

IF arrow mode is enabled

 Disable player movement

 Draw the arrow

ELSE

 Remove the arrow

 Enable player movement

END IF

METHOD: actionLogic

This will contain the logic for the action listener so the action listener will call this method and will run through this method to see what logic needs to be done if a certain action is taken. So, if a button is pressed.

When an action occurs

IF a piece is selected

 Record the selected piece

IF the piece is not set up

 Set up the piece interface

END IF

IF the piece is finished

 Deselect the piece

 Hide the piece interface

ELSE

 Show the piece interface

END IF

END IF

METHOD: actionPerformed

If an action is triggered, then it will call actionLogic to do the logic for that action

CLASS: SelectFile

This is used so when the user can select the file for the board to load.

METHOD: setUpForInput

Creates the FileNameExtention and the panel to put it on and a way to close it safely without it breaking and give the fileChoser a action listener then sets the panel as visible. Then makes setup equal to true and closed set as false.

Configure the file chooser to accept text files only

Configure the text input window

Keep the window on top

Allow resizing

Set window size

Center the window on screen

Close the window when dismissed

When the window is closed

Mark the window as closed

END WHEN

Register the current object to handle file chooser actions

Add the file chooser to the interface

Display the text input window

Mark setup as complete

Reset the closed status

METHOD: getText

This is a getter to get the value of the variable.

METHOD: isSubmitted

This is a getter to get the value of the variable.

METHOD: isSetUpDone

This is a getter to get the value of the variable.

METHOD: isClosed

This is a getter to get the value of the variable.

METHOD: reset

This class resets the variables all to their default state.

METHOD: actionPerformed

This checks what the fileChosser has returned and if it is a valid file and return the file selected or if it should close the window

Get the user's action command

IF the user approves the file selection

Mark the input as submitted

Store the selected file location

Mark the window as open

Close the text input window

ELSE IF the user cancels the selection

Mark the input as not submitted

Mark the window as closed

Close the text input window

END IF

CLASS: SaveBoardState

This is used to select the area and the file name to save the board you have created.

METHOD: setUpForInput

Creates the fileChooser and puts it into save dialog mode

METHOD: convertToString

This method takes 2D array of characters and converts it to a string to save into a file and converts it to BNF like format.

Create an empty position string

Set an empty-square counter to zero

FOR each row of the board

FOR each square in the row

IF the square is empty

Increase the empty-square counter

ELSE

IF there were empty squares before

Add the count to the position string

END IF

Add the piece symbol to the position string

Reset the empty-square counter

END IF

END FOR

IF the row ends with empty squares

Add the count to the position string

END IF

Separate rows in the position string

Reset the empty-square counter

END FOR

Remove the final row separator

Return the position string

METHOD: getName

This is a getter to get the value of the variable.

METHOD: isSubmitted

This is a getter to get the value of the variable.

METHOD: isSetUpDone

This is a getter to get the value of the variable.

METHOD: isClosed

This is a getter to get the value of the variable.

METHOD: reset

This class resets the variables all to their default state.

METHOD: actionPerformed

Checks what the use clicks on the filechooser and if its valid and then acts according for what the user clicks. Either it closes the program or approves the file and gets it path.

CLASS: BoardHolder

This is the class used to contain all the stuff needed to work with the board and what the user sees like the top menu and the side menu as well as the main board.

METHOD: BoardHolder

Creates the main board used and passes it FramePanel

METHOD: checkSelectFile

Calls the topMenu to check is selectFile has been pressed

METHOD: saveFileChecker

Calls the topMenu to check is saveFile has been pressed

METHOD: getSelectedPiece

Calls the topMenu to check is selectPiece has been pressed

METHOD: checkSwitchSides

Calls the topMenu to check is SwitchSides has been pressed

METHOD: selectFileOff

Tells topMenu to set selecFile button to false

METHOD: turnOffSaveFileChecker

Tells topMenu to set saveFile button to false

METHOD: switchSidesOff

Tells topMenu to set switchSdes button to false

METHOD: turnOffselectPiece

Tells topMenu to setselectPiece button to false

METHOD: flipBoard

Calls the main board flip method to flip the board

METHOD: getLastButtonAsText

Ask the main board to get the last button presses on the board and set the button text as the text given in the method

METHOD: setButtonArray

Sets the chess button array as the one that is imputed into the method.

METHOD: getButtonArrayJPanel

Returns a JPanel of the button array

METHOD: loadBoard

Runs the loadBoard method in the mainboard class

METHOD: getBoardState

Returns a 2D array of the current board that is displayed

METHOD: setLetMove

Takes a Boolean and then gives it to the chess board setMove Function to see if the pieces can move

METHOD: checkArrowMode

Calls topMenu to see if ArrowMode is active

METHOD: arrowDrawing

Calles arrowLogics getXAndYOfButtonInPixels method and takes the last button pressed and the board for the arrowLogic Method.

METHOD: erraseArrow

Calls arrowLogic erraseArrow method to erase the arrow that is draw on the board.

METHOD: getBoardPanel

Returns the JPanel used full of the board

METHOD: returnBoard

Takes and returns a JFrame. This method creates the board with layed panels and a set screen width and hight and sets up the JFrame for use by adding all the necessary components needed.

Create the main board panel

Get the screen dimensions

Set the board size to match the screen

Configure the main board window

Close the application when the window closes

Display the window

Create a layered interface

Place the board on the base layer

Place arrow graphics above the board

Make the arrow layer transparent

Set the layered interface size to match the window

Resize the board and arrow layers to fit the playing area

Add all interface components to the board

Place the layered interface in the center

Place the menu at the top

Place number labels on the left

Place letter labels at the bottom

Return the configured board

CLASS: SelectPiece

Creates the panel of pieces used to select when building a board.

METHOD: setUpPanel

Creates the JFrame and Panel of the board and adds the panel text of the pieces on it

METHOD: getName

Returns the name of the piece that has been selected

METHOD: hidePanel

Sets the visibility of the JFrame to false

METHOD: showPanel

Sets the visibility of the JFrame to true

METHOD: getDone

Return if the done button has been clicked to close the panel

METHOD: getSetUp

Return if the window set up has been completed

METHOD: setIcons

Sets the icons of the panel with images using a switch case

METHOD: actionLogic

Performs the action logic for the class to get which button is pressed and then get the text from it

METHOD: actionPerformed

Calls action Logic with the event for the logic.

FOR each relevant square on the board
 Read the piece identifier from the square

 IF a piece is present
 Choose the correct image for that piece
 Display the image on the square

 END IF
END FOR

CLASS: FileHandler

Holds everything to do with the actual files in the project and has it all in one class

METHOD: readFile

Calls fileRead with a string to read that file

METHOD: getData

Readers fileRead data that was read from the above method

METHOD: turnDataToArray

Gets the data from the method above and puts it into dataToArray inputting method to turn it into a array

METHOD: returnChessArray

Returns a 2D array of characters that was made In the method above

METHOD: dataToSave

Takes a file name and a board state both as strings and then uses file maker dataToSave method to save the file.

CLASS: ArrowLogic

Does everythink to draw and erase the arrow and get the positions.

METHOD: paintComponent

Is the paint component that is used to easily to draw the arrow

METHOD: drawArrow

Does the maths to draw the arrow and sets it colour with paint component

Set color to red
Set stroke thickness

Draw line from (startX, startY) to (endX, endY)

Compute direction = angle from start to end

Set arrowhead size

Create arrowhead shape at (endX, endY)

Rotate arrowhead by direction

Draw and fill the arrowhead

Reset drawing transformations

METHOD: getXAndYOfButtonInPixels

Gets the x and y of the buttons and stores them

```
IF first position is not set
  FOR each row in board
    FOR each column in board
      IF the clicked button matches this square
        Set first position to the center of this square
        gotBothPos = FALSE
      END IF
    END FOR
  END FOR
ELSE
  FOR each row in board
    FOR each column in board
      IF the clicked button matches this square
        Set second position to the center of this square
        Set width and height of the square
        gotBothPos = TRUE
      END IF
    END FOR
  END FOR
END IF
```

METHOD: checkIfBothPosHave

Returns a Boolean value of gotBothPos which is true if both values for the arrow positions to draw.

METHOD: eraseArrows

Sets all the values to their default value and removes the arrows positions

CLASS: TopMenu

This is the class that contains all the buttons that the user can press in the top menu

METHOD: TopMenu

Takes a integer and then sets it as the textSize used in topMenu button text Size

METHOD: topMenuMaker

Creates the JPanel and creates the buttons adds the action Listener for it with the buttons fonts and backgrounds and adds it to the panel.

METHOD: turnOffSaveFileChecker
Turns the Boolean variable saveFileChecker to false

METHOD: turnOffSelectPiece
Turns the Boolean variable getSelectPiece to false

METHOD: selectFileOff
Turns the Boolean variable selectFileCheck to false

METHOD: switchSidesOff
Turns the Boolean variable runSwitchSides to false

METHOD: getSelectPiece
Returns the Boolean value of getSelectPiece

METHOD: checkSaveFileChecker
Returns the Boolean value of saveFileChecker

METHOD: checkSelectFile
Returns the Boolean value of selectFileCheck

METHOD: checkSwitchSides
Returns the Boolean value of runSwitchSides

METHOD: checkArrowMode
Returns the Boolean value of arrowModeOn

CREATE a new panel

CREATE button "Select File"
Set background color to white
Set font size
Register action listener
Add button to panel

CREATE button "Save State"
Set background color to white
Set font size
Register action listener
Add button to panel

CREATE button "Pieces"
Set background color to white
Set font size
Register action listener
Add button to panel

CREATE button "Switch Sides"
Set background color to white
Set font size
Register action listener
Add button to panel

CREATE button "Arrow Mode"
Set background color to white
Set font size
Register action listener
Add button to panel

RETURN the panel

METHOD: actionLogic
Does the logic for action performed. If a button is selected it turns that buttons Boolean variable to true.

METHOD: actionPerformed
Runs actionLogic with the action that triggered this as the input

CLASS: PanelForButtonArray
This holds the chess board array and everything needed to be needed with the chess board.

METHOD: PanelForButtonArray
Starts the timer and gives the class the JFrame to be use in the board.

METHOD: buttonArray
Creates the panel used in the button array and adds the buttons to it. Then loads the board.

METHOD: lastButtonPresed

When a event is made this will store the button that caused the action.

METHOD: setIcons

Uses a switch case to load the images onto the tile.

METHOD: actionPerformed

Sets the lastAcrtionEvent as the event that triggered this then runs moves pieces with the actionEvent selected and runs lastButtonPressed with the input of the event that triggered this. Then finally it sets the icons of all the pieces.

METHOD: loadBoard

Goes through the whole chess array. Sets there icons as null and then sets the text of the button as the chessArrays text at that position of the array.

```
FOR each row in the relevant board area
  FOR each column in the relevant board area
    Get the piece symbol from the square

    SELECT the piece symbol
      CASE "P":
        Set the square's image to white pawn
      CASE "R":
        Set the square's image to white rook
      CASE "N":
        Set the square's image to white knight
      CASE "B":
        Set the square's image to white bishop
      CASE "Q":
        Set the square's image to white queen
      CASE "K":
        Set the square's image to white king
      // Add other piece cases as needed
    END SELECT
  END FOR
END FOR
```

METHOD: setLetMove

It takes a Boolean variable and sets letMove to that variable given

METHOD: getActionEvent

Returns the lastEvent

METHOD: setButtonArray

Takes a 2D array of characters and sets it as the chessArray

METHOD: getButtonArray

Returns the 2D array of JButtons

METHOD: getLastButton

Returns the last Button Pressed

METHOD: getBoardState

Takes the chess board and takes the text of it and puts it to a array that is then return from the method

METHOD: resetColourOfButton

Gets the last button pressed and then resets it to its old colour.

METHOD: movePieces

This is the class to move the pieces. When an action is done this method is called and if letMove is true then it searches for the button that pressed it. If it is a piece then it will store it and remove the icon when moving to the next button.

```
IF a piece is being moved AND the current square is not empty
  Store the piece from the square
  Remove the piece's image
  Remember the old coordinates
  Mark movePiece as FALSE
  Clear the text from the square
  Reset the square's background color
ELSE IF no piece is currently being moved
  Store the current square's background color
  Remember the coordinates of the square

  IF the move from the old position to this square is valid
    Highlight the square green
  ELSE
    Highlight the square red
  END IF

  Set the square's text to the piece
  Mark movePiece as TRUE
END IF
```

METHOD: flipBoard

Flips the whole board in the x and y axis. So, the pieces are flipped in so you can see the pieces for the opponent's side.

```
CREATE a new 8x8 flipped button array

FOR each square on the board
  Clear the square's image
END FOR

FOR each square in the flipped array
  Initialize a new button
END FOR

// Flip the board along both axes
FOR each row in the board
  FOR each column in the board
    Set the flipped array's square to the text from the mirrored square of the original board
  END FOR
END FOR

// Copy the flipped board back to the original
FOR each row in the board
  FOR each column in the board
    Update the original board's square with the flipped square's text
  END FOR
END FOR

Reset the colors of all buttons
```

CLASS: SidePartsOfBoard

Contains the side of the board with all the numbers and letters

METHOD: SidePartsOfBoard

Takes a integer and then sets it as the textSize used in topMenu button text Size

METHOD: Numbers

Creates a JPanel with a set of JButtons using a grid layout. It returns the JPanel. This panel has the numbers as the set Text

METHOD: Letter

Creates a JPanel with a set of JButtons using a grid layout. It returns the JPanel. This panel has the letters as the set Text

METHOD: flipNumbersAndLetters

Flips both the letters and number panel text so 1-8 becomes 8-1 and the same for the letters

CLASS: FileMaker

Is the class to make the file for them to be saved

METHOD: dataToSave

Takes a name and data as inputs off Strings and then runs createFile with the name and wrteDataToFile to save the data into the file

METHOD: createFile

Creates the file with the file object and then saves it

METHOD: writeDataToFile

Gets the path of the file and then opens it and writes the data into the file.

FUNCTION createFile(name)

TRY

Create a file object with the given name

IF the file does not already exist

Create the file

RETURN TRUE // file successfully created

ELSE

RETURN FALSE // file already exists

END IF

CATCH any error

Print error details

RETURN FALSE // error occurred

END TRY

END FUNCTION

FUNCTION writeDataToFile(name, data)

Convert the data to bytes

TRY

Write the bytes to the file with the given name

RETURN TRUE // write successful

CATCH any error

Print error details

RETURN FALSE // write failed

END TRY

END FUNCTION

CLASS: DataToArray

Gets the strings and then turns them to the chess array from BNF format to a 2D character array

METHOD: Inputing

Takes each charter and converts it to the 2D array

METHOD: returnChessArray

Returns the chess array

METHOD: boardStateOutput

Returns a string version of the chess array

```
Initialize nextLineCount to 0
Initialize xAxis to 0
Initialize skipRest to FALSE
Initialize positionData to empty

FOR each character in the input data
  IF character is a space
    skipRest = TRUE

  ELSE IF character is a digit AND skipRest is FALSE
    repeatCount = numeric value of character
    FOR repeatCount times
      Set chessBoardArray[xAxis][nextLineCount] to empty
      xAxis += 1
    END FOR

  ELSE IF skipRest is TRUE
    Append character to positionData

  ELSE IF character is not a digit AND skipRest is FALSE
    IF character is '/'
      nextLineCount += 1
      xAxis = 0
    ELSE
      Set chessBoardArray[xAxis][nextLineCount] to character
      xAxis += 1
    END IF
  END IF
END FOR
```

CLASS: FileReader

The class that's reads the data from the file

METHOD: readFile

Takes a file name and reads the data out of it.

METHOD: dataSpliter

Splits up the data and adds it to the the DataString

METHOD: getData

Returns the data string

CLASS: ValidMoveGetter

Class to see if a move is valid or not

METHOD: checkIfValidMove

Uses maths to get the diffrents of the moves and then uses the piece to get to see if the move is valid or not.

```
Compute xdiff = absolute difference between  
lastX and newX  
Compute ydiff = absolute difference between  
lastY and newY
```

```
Convert xdiff and ydiff to absolute values
```

```
SELECT the piece type (uppercase)
```

```
  CASE "P": // Pawn
```

```
    IF the pawn is on its starting row
```

```
      IF xdiff is 1 OR xdiff is 2
```

```
        RETURN TRUE // valid move
```

```
      END IF
```

```
    ELSE
```

```
      IF xdiff is 1
```

```
        RETURN TRUE // valid move
```

```
      END IF
```

```
    END IF
```

```
  END CASE
```

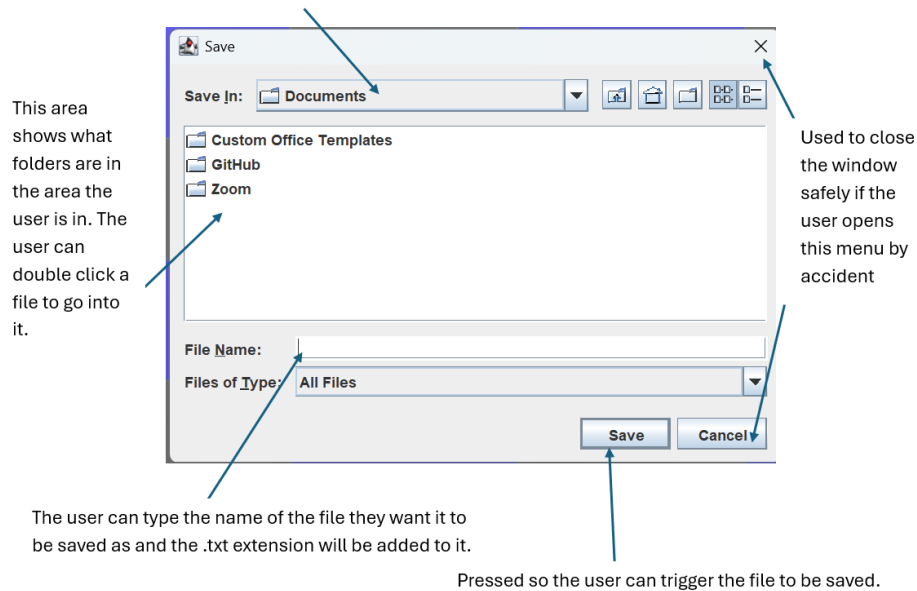
```
  // Other piece cases would go here
```

```
END SELECT
```

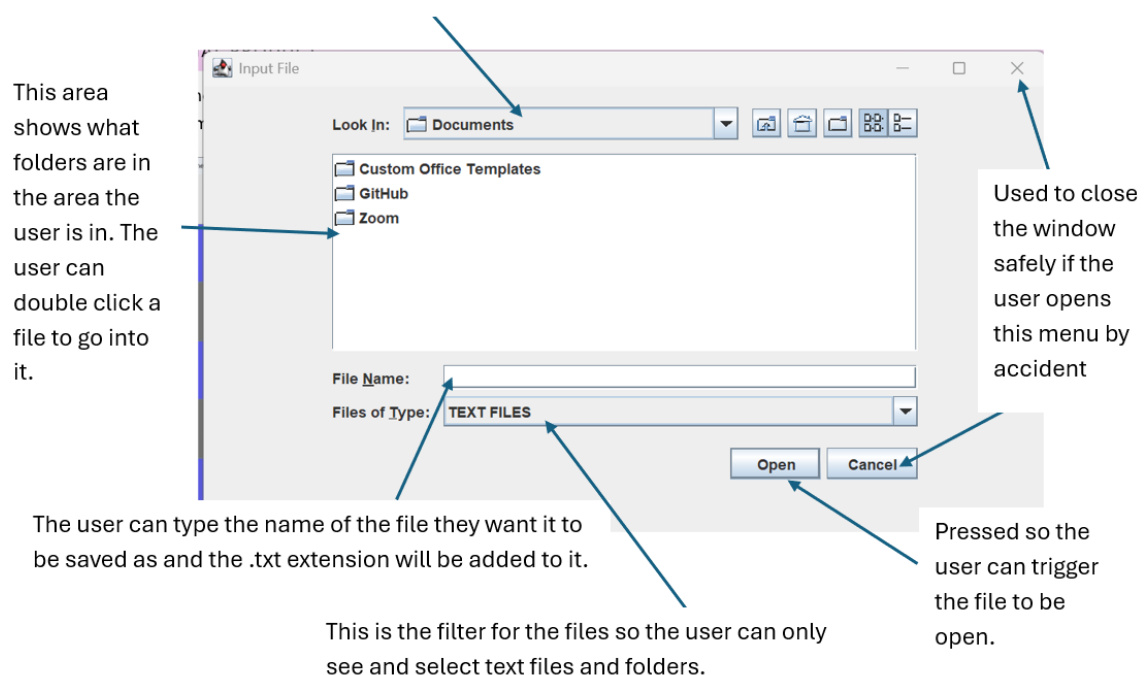
TECHNICAL SOLUTION

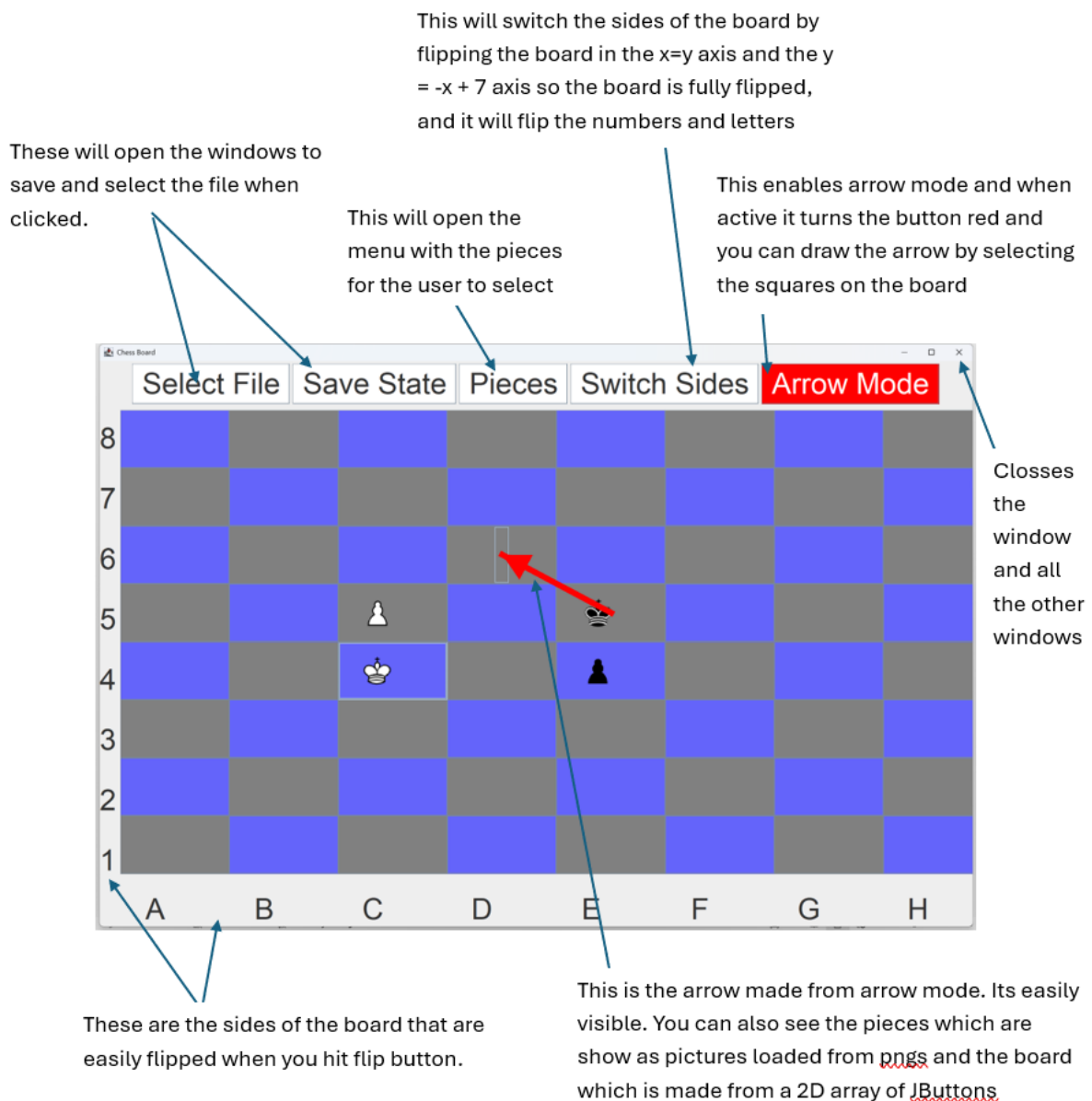
FINAL PRODUCT

This is the Folder that the user is currently in. The user can click this window to get a drop-down menu to select the folder the user wants to go into.

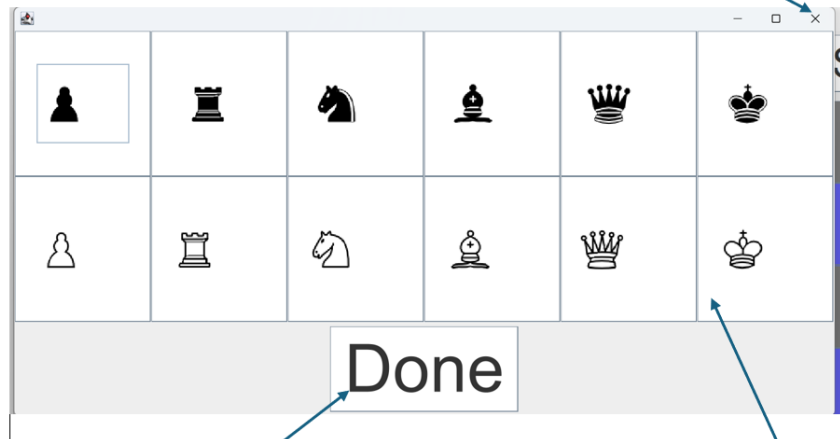


This is the Folder that the user is currently in. The user can click this window to get a drop-down menu to select the folder the user wants to go into.





This will NOT close the window but bring it back to its default form as the done button needs to be clicked so the board is set back to be used safely



This will close the window and stop the board from not being able to move.

These are the buttons with the pieces in them. When the user clicks the buttons then the piece the button on is selected and then the user clicks a square on the board and that piece that was selected will also be at the square selected by the user.

CODE SEGMENTS

ANYTHINK COMPECT

CLASS: FramePanel

Logic used for showing save and select windows

```

if(boardHolder.saveFileChecker()){
    if(!saveBoardState.isSetupDone()){
        saveBoardState.setUpForInput();
    }

    if (saveBoardState.isSubmitted()) {
        dataOfBoardInNFS = saveBoardState.convertToString(boardHolder.getBoardState());
        fileNameToSaveAs = saveBoardState.getName();
        fileHandler.dataToSave(fileNameToSaveAs, dataOfBoardInNFS);

        boardHolder.turnOffSaveFileChecker();
        saveBoardState.reset();
    }

    if(saveBoardState.isClosed()){
        boardHolder.turnOffSaveFileChecker();
        saveBoardState.reset();
    }
}

.f(boardHolder.checkSelectFile()){
    if(!selectFile.isSetupDone()){
        selectFile.setUpForInput();
    }

    if(selectFile.isSubmitted()){
        fileHandler.readFile(selectFile.getText());
        fileHandler.turnDataToArray();

        boardHolder.setButtonArray(fileHandler.returnChessArray());
        boardHolder.loadBoard();

        boardHolder.selectFileOff();
        selectFile.reset();
    }

    if(selectFile.isClosed()){
        boardHolder.selectFileOff();
        selectFile.reset();
    }
}

```

Gets the board and converts it to a string and saves it to a variable to parse it later and it also gets the files name from the FileChooser that the user typed to use as the name which also includes its file path. Then it is parsed into the method to create and save the file of that name. This happens when the user clicks the submit button

Turns off the save board and the topMenu so this loop doesn't trigger when the user is finished.

When the select File mode is made and setup is done is has not happen the panel will be created from running this method

When the user has done what they need and the file has been created then the select file will make closed true making this true and resting the whole class for the user to run it again so they can save a new board.

CLASS: SelectFile

Makes the JFileChooser and sets it up.

This sets up a filter for JFileChooser so the user can only pick text Files (*.txt)

```
public void setUpForInput() {
    FileNameExtensionFilter filter = new FileNameExtensionFilter("TEXT FILES", "txt", "text");
    fileChooser.setFileFilter(filter);

    textInput.setAlwaysOnTop(true);
    textInput.setResizable(true);
    textInput.setSize(700, 400);
    textInput.setLocationRelativeTo(null);
    textInput.setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);

    textInput.addWindowListener(new java.awt.event.WindowAdapter() {
        @Override
        public void windowClosed(java.awt.event.WindowEvent e) {
            closed = true;
        }
    });

    fileChooser.addActionListener(this);

    textPanel.add(fileChooser);
    textInput.add(textPanel);

    textInput.setVisible(true);

    setUpDone = true;
    closed = false;
}
```

Makes a JFrame that's always on top to display the JFileChooser with the set size at the middle of the screen that gets disposed on closed

This makes the frame adds a windows Listener it overrides the windows Closed method in the windows Listener and sets that to so closed equal true. Which will close the JPanel the way I need

Adds an action Listener to the JFileChooser then adds the fileCooser to the JPanel that is then added to the JFrame. The JFrame is then set as visible and the setUpDone is set to true as well as closed is set to false

Handles when a user triggers an action in the JFileChooser

```
@Override
public void actionPerformed(ActionEvent e) {

    String cmd = e.getActionCommand();

    if (cmd.equals(JFileChooser.APPROVE_SELECTION)) {
        submitted = true;
        text = fileChooser.getSelectedFile().getAbsolutePath();
        closed = false;
        textInput.dispose();
    }

    else if (cmd.equals(JFileChooser.CANCEL_SELECTION)) {
        submitted = false;
        closed = true;
        textInput.dispose();
    }
}
```

Gets the actionEvent that triggered this and stored it as a string

If the actionEvent equals the approve section from JFileChooser then submitted is set to true and then the string text is set as the path to the selected file and closed is false and then the JFrame is disposed of.

If the actionEvent equals the cancelSelection on the JFileChooser where the user presses the cancel button, then submitted is set to false and closed is true which will close this window and then the JFrame is disposed of.

CLASS: SaveBoardState

When ran this will set up the JFileChooser ready to be seen by the user

This sets up the variables with their default values and then removes the `actionListener` from `fileChooser` in case it already has one from if it ran before. Then we add a new `actionListener` back.

```
public void setUpForInput() {
    submitted = false;
    closed = false;
    setupDone = true;

    fileChooser.removeActionListener(this);
    fileChooser.addActionListener(this);

    int result = fileChooser.showSaveDialog(null);

    if (result != JFileChooser.APPROVE_OPTION && !submitted) {
        closed = true;
    }
}
```

`fileChooser.showSaveDialog` will show the save version of the `fileChoser` and we set that to the `result`

This checks if the result is a approve option and that the answer is submitted if so, it sets it closed as true and closes this frame.

This converts the 2D array to a string in BNF format

Creates a nested for loop so it can go through the full 2D array of the list given.

```
public String convertToString(char[][] boardState) {
    positionData = "";
    int count = 0;

    for (int y = 0; y < 8; y++) {
        for (int x = 0; x < 8; x++) {

            if (boardState[x][y] == ' ') {
                count++;
            } else {
                if (count != 0) positionData += count;
                positionData += boardState[x][y];
                count = 0;
            }

            if (count != 0) positionData += count;
            positionData += "/";
            count = 0;
        }

        return positionData.substring(0, positionData.length() - 1);
    }
}
```

Then at each cell of the array it checks if it's a space and if it is then count is incremented by one else it checks if the count is zero if no then count is added to `positionData` then the cell is added from the 2D array at the point to position data. Count is then reset.

Then in between the nested for loops it checks if count is above zero and added that many to `positionData` if true after that a / is added to symbolize a new line and count is reset again. This is all done to put the array to BNF format

The string is then returned with the last line character removed so the last / is no in the string sent back

Handles when a user triggers an action in the JFileChooser (This is the same as the selectFile as they are doing the same thing needed at this point in both classes)

```
@Override
public void actionPerformed(ActionEvent e) {

    String cmd = e.getActionCommand();

    if (cmd.equals(JFileChooser.APPROVE_SELECTION)) {
        submitted = true;
        text = fileChooser.getSelectedFile().getAbsolutePath();
        closed = false;
        textInput.dispose();
    }

    else if (cmd.equals(JFileChooser.CANCEL_SELECTION)) {
        submitted = false;
        closed = true;
        textInput.dispose();
    }
}
```

Gets the actionEvent that triggered this and stored it as a string

If the actionEvent equals the approve section from JFileChooser then submitted is set to true and then the string text is set as the path to the selected file and closed is false and then the JFrame is disposed of.

If the actionEvent equals the cancelSelection on the JFileChoser where the user presses the cancel button, then submitted is set to and closed is true false which will close this window and then the JFrame is dispose of.

TESTING

A test should be designed for each success criteria, and should be enough to evidence that the program fully works and meets all the success criteria

TEST TABLE

Test Number	Success Criteria	Description	Expected outcome	Observed outcome	Remedial action (if necessary)
1	1	Launch application, and start 1 player game	First level loads with a single yellow pacman	Game successfully started with one player	None needed
2	1	Launch application, and start 2 player game	First level loads with yellow and green pacman		
3	2	On menu screen, press left to change p1 colour and "a" to change p2 colour and start game	Both players should be their new colours		
4					

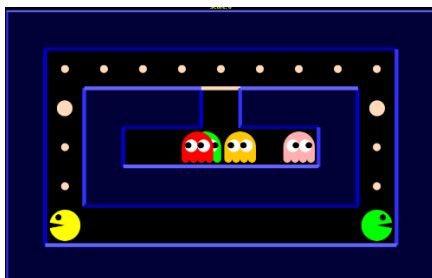
EVIDENCE

Screenshot evidence where appropriate for each test

TEST 1



TEST 2



EVALUATION**ANALYSIS OF CRITERIA**

Include each of your success criteria along with a description (and reference to evidence if possible) with how you met it

END USER FEEDBACK

Include some feedback from your end user and analyse it to explain how successful your product was

FURTHER DEVELOPMENT

If the product were to continue to be developed, how could this be done? What could be added or improved?

FINAL THOUGHTS

Overall, how do you feel about your product and the experience of making it? (be positive)